제9장 이미지 인식과 CNN — 컴퓨터가 이미지를 보는 방식

학습 목표

- 이미지 한 장이 컴퓨터에게 어떤 숫자 덩어리인지 설명한다
- 픽셀과 채널을 구분하고, 흑백 이미지와 컬러 이미지가 숫자 판 몇 장인지 말한다
- 이미지를 한 줄로 펴서 넣으면 무엇이 사라지는지 실험 결과로 설명한다
- 3x3 필터로 합성곱을 손으로 한 칸 계산하고, 필터가 어떤 창에 큰 값을 내는지 말한다
- pooling이 크기를 줄이면서 무엇을 남기고 무엇을 남기지 못하는지 설명한다
- CNN의 층 순서를 그리고, 필터의 숫자를 학습으로 찾는다는 뜻을 설명한다

지금까지 다룬 데이터는 표였다. 가로줄 하나가 학생 한 명이었고, 세로줄 하나가 공부 시간이나 수면 시간 같은 특징이었다. 이번 장에서는 이미지를 다룬다. 이미지도 결국 숫자지만, 표와 달리 숫자가 **어느 칸에 놓였는지**에 뜻이 있다. 합성곱은 칸의 위치 관계를 살려서 계산하는 방법이고, 합성곱을 층으로 쌓은 신경망이 CNN이다.

이 장은 문제를 먼저 측정하고 해법을 뒤에 둔다. 1~3절에서 이미지를 숫자로 보고 한 줄로 펴서 넣는 방식을 설명한 뒤, 실습 9.1에서 그 방식으로 분류기를 만들어 어디까지 되고 어디서 무너지는지 측정한다. 4~5절에서 합성곱과 pooling을 설명하고, 실습 9.2에서 둘을 직접 계산한다. 6절에서는 이 계산을 층으로 쌓은 CNN의 구조를 그림으로 본다.

○ 합성곱을 직접 계산하는 이유

- 딥러닝 라이브러리는 합성곱을 함수 하나로 계산해 주므로, 안에서 무슨 일이 일어나는지 보이지 않는다
- 이 장의 실습은 합성곱과 pooling을 numpy 반복문 두 개로 직접 계산하고, 필터의 숫자 아홉 개를 직접 정해 넣는다. 숫자를 바꾸면 결과가 어떻게 바뀌는지 칸 단위로 따라갈 수 있다

실행 환경

```
cd practice/chapter9
python -m pip install -r ../requirements.txt
```

필요한 패키지는 numpy, pandas, matplotlib, scikit-learn이다. GPU는 필요 없다. 두 실습 모두 노트북에서 몇 초 안에 끝난다.

두 실습의 데이터는 모두 `sklearn.datasets.load_digits()`다. 합성 데이터가 아니라 실제 손글씨를 스캔해 만든 공개 데이터이고, scikit-learn 안에 들어 있어서 따로 내려받지 않는다.

1. 이미지는 숫자다

이미지 파일을 열면 사람은 그림을 보고, 컴퓨터는 숫자를 받는다. 이 장에서 쓰는 데이터는 손글씨 숫자 이미지이고, 한 장의 크기는 8 x 8 픽셀이다.

```
scikit-learn load_digits() - 실제 손글씨를 스캔해 만든 공개 데이터
이미지 장수: 1797장
이미지 한 장의 크기: 8 x 8 픽셀
채널 수: 1 (흑백이므로 숫자 판이 한 장이다)
픽셀값 범위: 0 ~ 16
이미지 한 장이 숫자 64개다
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

17번 이미지의 픽셀값을 그대로 찍으면 다음과 같다.

```
        0    1    2    3    4    5    6    7
    ----------------------------------------
0 |    0    0    1    8   15   10    0    0
1 |    0    3   13   15   14   14    0    0
2 |    0    5   10    0   10   12    0    0
3 |    0    0    3    5   15   10    2    0
4 |    0    0   16   16   16   16   12    0
5 |    0    1    8   12   14    8    3    0
6 |    0    0    0   10   13    0    0    0
7 |    0    0    0   11    9    0    0    0
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

이 숫자 64개가 이미지의 전부다. 0이 검정이고 16이 흰색이다. 4번 줄에서 16이 네 번 이어지는 칸이 밝은 가로획이다. 이 이미지의 정답은 7이다.

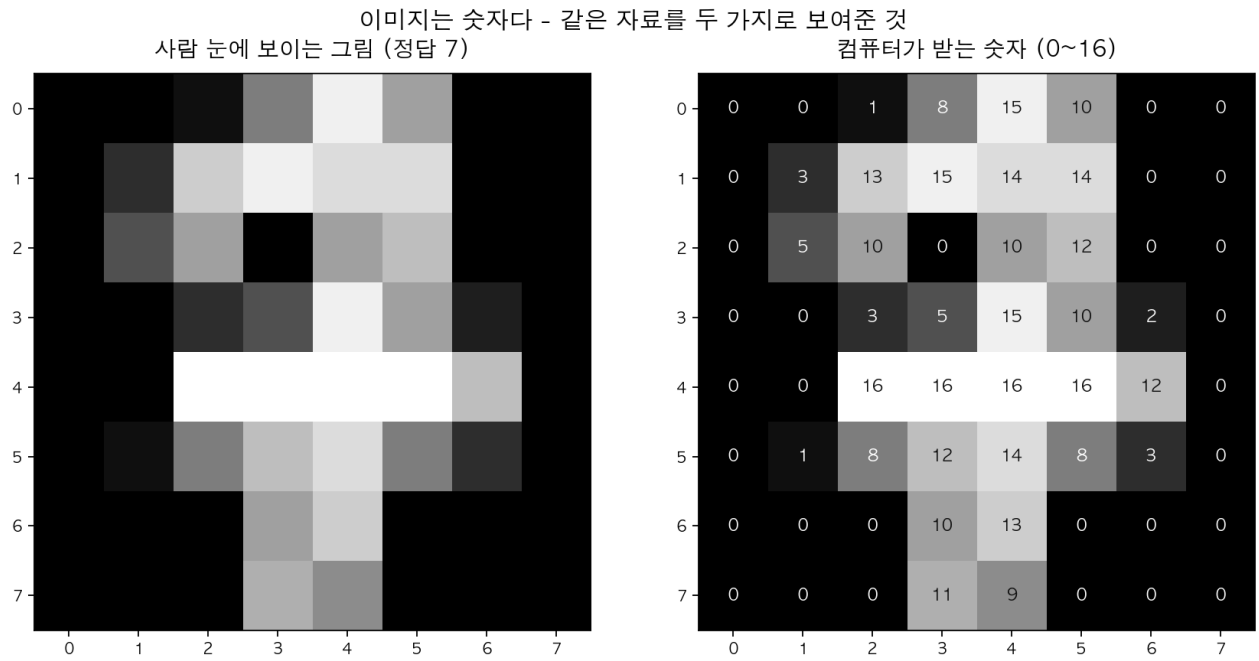


그림 9.1 왼쪽과 오른쪽은 같은 자료다. 사람은 왼쪽을 보고, 컴퓨터는 오른쪽을 받는다

○ **여기서 확인할 것**

- 사람이 "7처럼 생겼다"고 느끼는 그림과 컴퓨터가 받는 숫자 64개는 같은 자료다
- 밝기 차이가 곧 숫자 차이다. 획이 지나간 칸은 값이 크고, 배경 칸은 0이다

---

2. 픽셀과 채널

○ **픽셀(pixel)** — 이미지를 이루는 가장 작은 칸 하나

- 8 x 8 이미지에는 픽셀이 64개 있다
- 흑백 이미지에서 픽셀 하나는 밝기를 뜻하는 숫자 하나다

○ **채널(channel)** — 같은 크기의 숫자 판이 몇 장 겹쳐 있는가

- 흑백은 판이 한 장이므로 채널 수가 1이다
- 컬러는 빨강·초록·파랑 판이 한 장씩 있어서 채널 수가 3이다

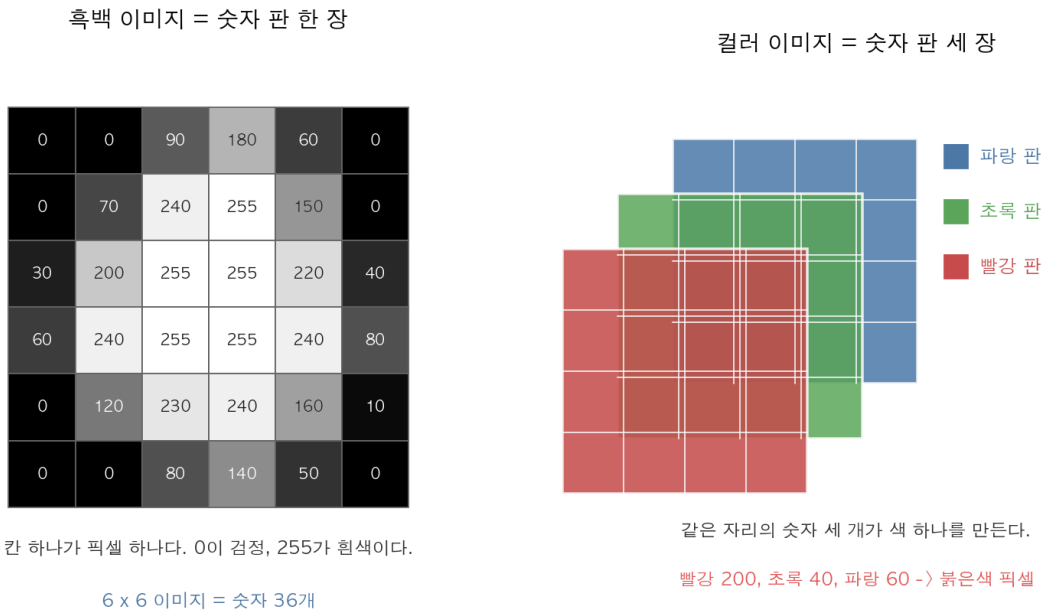


그림 9.2 같은 위치에 있는 세 판의 숫자 세 개가 색 하나를 만든다

컬러 이미지에서 픽셀 하나의 색을 알려면 숫자를 세 개 봐야 한다. 빨강 판에서 200, 초록 판에서 40, 파랑 판에서 60이면 그 픽셀은 붉은색이다.

이미지가 커지면 숫자 개수는 가로 × 세로 × 채널 수만큼 늘어난다.

표 9.1 이미지 크기와 숫자 개수

이미지	채널	숫자 개수
8 x 8 흑백 (실습 데이터)	1	64
28 x 28 흑백	1	784
200 x 200 컬러	3	120,000
1000 x 1000 컬러	3	3,000,000

---

3. 한 줄로 펴서 넣으면 무엇이 사라지는가

5장에서 만든 신경망은 입력을 한 줄로 받았다. 이미지를 그 신경망에 넣으려면 8 x 8을 한 줄로 펴야 한다. 위 줄부터 차례로 이어 붙이면 숫자 64개짜리 한 줄이 된다. 이 작업을 **평탄화(flatten)**라고 부른다.

○ **영상으로 먼저 보기** — [딥러닝 프로젝트] 1강. 합성곱 신경망(CNN) (한빛미디어, 약 11분)

- 주소: https://www.youtube.com/watch?v=hSUOpeUKqrM
- 영상 설명란의 목차는 00:43 인공 신경망, 03:45 합성곱 신경망, 09:47 풀링층이다
- 3절에서는 00:43의 인공 신경망 부분을, 4~5절을 읽은 뒤에는 03:45 이후를 이어서 본다

3-1. 위아래 이웃이 8칸 떨어진다

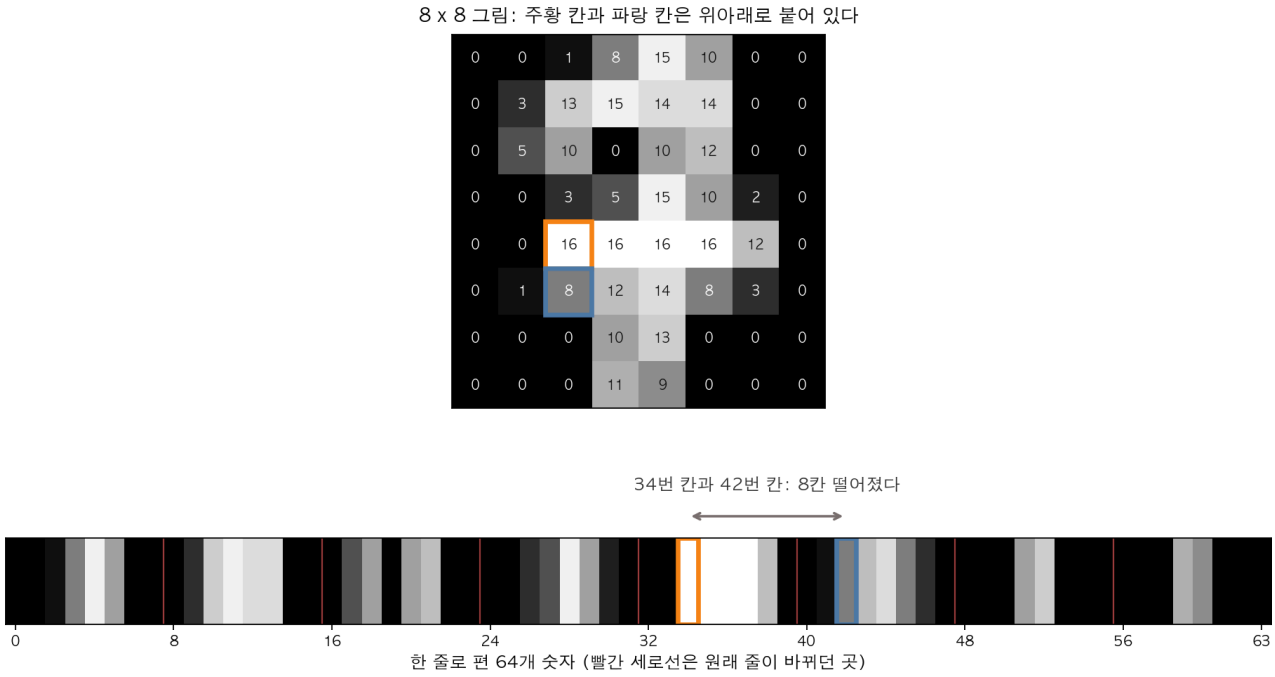


그림 9.3 주황 칸(4행 2열, 값 16)과 파랑 칸(5행 2열, 값 8)은 그림에서 위아래로 붙어 있다. 한 줄로 펴면 34번과 42번 자리로 가서 8칸 떨어진다

```
그림에서 위아래로 붙은 두 칸: (4행 2열, 값 16)과 (5행 2열, 값 8)
한 줄로 편 뒤의 자리: 34번과 42번 -> 8칸 떨어진다
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

가로로 붙은 두 칸은 한 줄에서도 붙어 있다. 위아래로 붙은 두 칸은 한 줄이 8칸이라서 8칸 떨어진다. 그림에서는 같은 획에 속한 두 칸이 한 줄에서는 멀리 놓인다.

3-2. 평탄화 모델은 칸 번호만 안다

평탄화 모델은 64칸을 0번부터 63번까지 번호로만 구별하고, 번호마다 가중치를 따로 둔다. "34번과 42번은 그림에서 위아래 이웃이다"라는 정보는 모델 어디에도 없다. 여기서 세 가지 결과를 예상할 수 있다.

○ 예측 1: 칸 순서를 섞어도 정확도가 거의 같다

- 모든 이미지의 칸 순서를 똑같이 섞으면 번호만 바뀌고 값은 그대로 남는다
- 모델은 바뀐 번호에 맞춰 가중치를 배운다. 사람은 섞인 그림을 알아볼 수 없지만 모델에게는 차이가 없다

○ 예측 2: 그림을 한 칸 밀면 정확도가 무너진다

- 7을 오른쪽으로 한 칸 옮겨 써도 사람 눈에는 같은 7이다
- 모델에게는 번호마다 들어오는 값이 바뀐다. 17번 이미지에서 34번 칸은 획의 밝기 16을 받았는데, 한 칸 밀면 배경의 0을 받는다

○ **문제 3: 이미지가 커지면 가중치가 너무 많아진다**

- 첫 층 뉴런 하나가 모든 칸과 연결되므로, 가중치 수는 입력 숫자 수 × 뉴런 수다
- 8 x 8 흑백에 뉴런 32개(실습 9.1의 모델)이면 64 × 32 = 2,048개다
- 200 x 200 컬러에 뉴런 100개이면 120,000 × 100 = 12,000,000개다

○ **바로 다음 실습 9.1의 설정**

- 1797장을 훈련 1257장과 테스트 540장으로 나누고, 8 x 8을 편 64개 숫자로 숫자 0~9를 분류한다
- 정확도와 자주 헷갈린 짝을 본 뒤, 예측 1과 예측 2를 실험으로 확인한다
- 실험 1은 64칸의 순서를 섞어 모델을 새로 학습한다. 실험 2는 학습이 끝난 모델에 한두 칸 민 테스트 이미지를 넣는다

실습 9.1: 64개 숫자로 분류하고 한계를 측정한다

실습 목표

- 8 x 8을 한 줄로 편 64개 숫자로 숫자 0~9를 분류하고, 기준선과 비교한다
- 틀린 이미지를 모아 실패 갤러리를 만들고, 어떤 짝이 자주 헷갈리는지 찾는다
- 칸 순서를 섞었을 때와 그림을 밀었을 때 정확도가 어떻게 달라지는지 측정한다

데이터와 모델

`load_digits()` 1797장이다. 모델은 scikit-learn의 `MLPClassifier`이고, 은닉층이 하나인 신경망이다. **CNN이 아니다.** 합성곱 층이 없다.

모델을 만드는 코드는 다음 한 줄이다.

```
MLPClassifier(hidden_layer_sizes=(32,), max_iter=60, random_state=42)
```

○ **은닉층(hidden layer)이란**

- 입력(숫자 64개)과 출력(숫자 0~9의 점수 10개) 사이에 있는 층이다. 입력과 출력은 우리가 정하는 값이고, 그 사이의 계산은 모델이 알아서 하므로 밖에서 보이지 않는다는 뜻에서 은닉(hidden, 숨겨진)이라고 부른다
- 은닉층의 뉴런 하나는 앞 층의 숫자를 전부 받아 하나씩 가중치를 곱해 더한 값 하나를 내보낸다

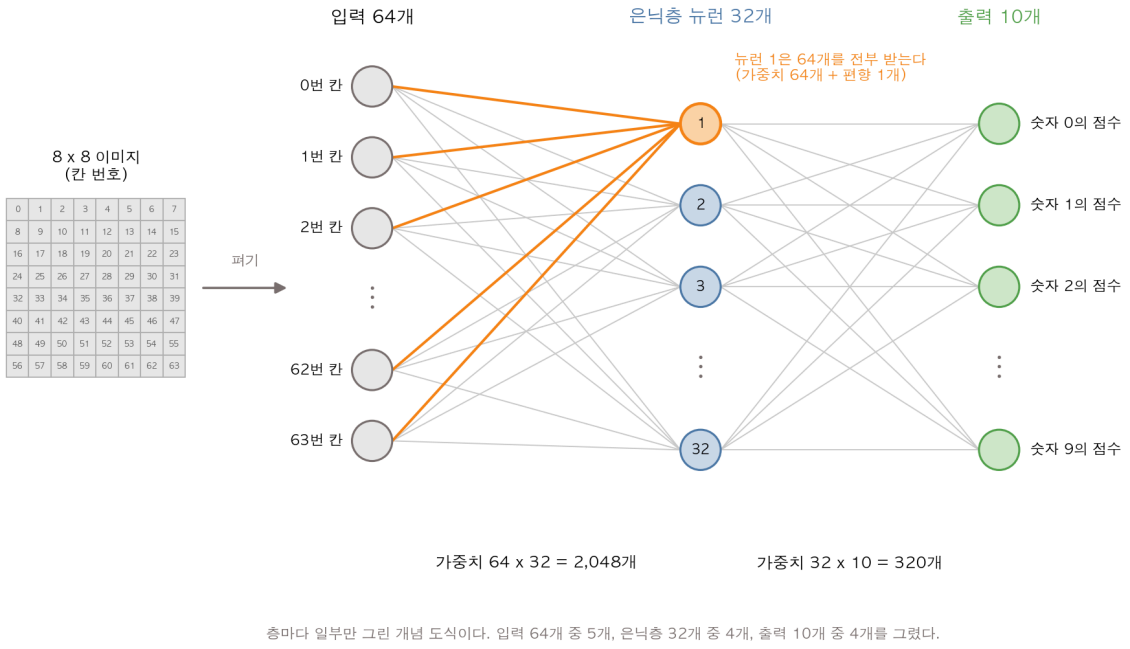
○ **`hidden_layer_sizes=(32,)`를 읽는 법**

- 괄호 안의 숫자 하나가 은닉층 하나이고, 숫자의 크기가 그 층의 뉴런 수다. `(32,)`는 "은닉층 1개, 뉴런 32개"다. 숫자가 하나뿐이어도 쉼표를 붙이는 것은 파이썬에서 원소가 하나인 묶음을 쓰는 규칙이다
- `(64, 32)`라고 쓰면 은닉층이 2개이고 뉴런 수가 64개, 32개다. 숫자를 늘리면 층이 깊어지고, 숫자를 키우면 층이 넓어진다

○ **이 모델의 크기를 숫자로 보면**

- 층별 뉴런 수는 입력 64 → 은닉 32 → 출력 10이다
- 연결마다 가중치가 있다. 입력과 은닉층 사이는 64 × 32 = 2,048개, 은닉층과 출력 사이는 32 × 10 = 320개다. 뉴런마다 따로 더하는 값(편향)이 은닉층 32개, 출력층 10개 있어서 학습으로 정하는 숫자는 모두 2,410개다
- 은닉층 뉴런 수를 바꾸면 이 개수가 바뀐다. 뉴런이 늘수록 모델이 만들 수 있는 모양이 다양해지고, 가중치도 그만큼 늘어난다. 7장 표 7.3에서 신경망의 복잡도 손잡이가 층 수와 뉴런 수라고 한 것이 이 값이다

그림 9.4는 이 모델을 층마다 몇 개만 골라 그린 것이다. 노드 106개를 모두 그리면 선이 겹쳐 읽을 수 없으므로 나머지는 점 세 개로 줄였다.



층마다 일부만 그린 개념 도식이다. 입력 64개 중 5개, 은닉층 32개 중 4개, 출력 10개 중 4개를 그렸다.

그림 9.4 실습 9.1 모델의 일부. 입력 64개 중 5개, 은닉층 뉴런 32개 중 4개, 출력 10개 중 4개만 그렸다. 주황 선은 은닉층 뉴런 1이 받는 연결이다

실행 명령

```
cd practice/chapter9/code
python 9-1-flatten-classifier.py
```

핵심 코드

파일 맨 위에 바꿀 값이 다섯 개 있다.

```
SAMPLE_INDEX = 17       # 픽셀 표로 볼 손글씨 번호. 0부터 1796까지 넣을 수 있다.
HIDDEN_SIZE = 32        # 은닉층 뉴런 수. 크게 하면 정확도가 오르지만 느려진다.
MAX_ITER = 60           # 학습을 몇 바퀴 돌 것인가.
TEST_RATIO = 0.3        # 테스트로 떼어 둘 비율
GALLERY_SIZE = 12       # 실패 갤러리에 넣을 이미지 장수
```

`HIDDEN_SIZE`는 코드 안에서 `hidden_layer_sizes=(HIDDEN_SIZE,)`로 들어간다. 이 값을 32에서 128로 바꾸면 입력과 은닉층 사이의 가중치는 64 × 128 = 8,192개가 된다.

실험 1은 64칸의 순서를 한 번 정해 훈련과 테스트 모두에 똑같이 적용한 뒤 새로 학습한다.

```
perm = np.random.default_rng(RANDOM_SEED).permutation(x.shape[1])
model_perm.fit(x_train[:, perm], y_train)
perm_acc = accuracy_score(y_test, model_perm.predict(x_test[:, perm]))
```

실험 2는 다시 학습하지 않는다. 테스트 이미지만 밀어서 원래 모델에 넣는다. 비는 칸은 0(검정)으로 채운다.

```
shifted = shift_images(img_test, dx, dy)
acc = accuracy_score(y_test, model.predict(flatten(shifted)))
```

전체 코드는 `practice/chapter9/code/9-1-flatten-classifier.py` 참고

실행 결과

```
전체 1797장 = 훈련 1257장 + 테스트 540장
입력: 8x8 이미지를 한 줄로 펴서 숫자 64개

기준선(가장 많은 숫자로만 답하기)              0.102
훈련 데이터                                  0.960
테스트 데이터                                 0.933
학습 반복 실제 횟수: 60회

테스트 540장 중 틀린 것 36장
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

```
hidden_layer_sizes = (32,)
층별 뉴런 수: 64 -> 32 -> 10  (입력 -> 은닉층 -> 출력)
1번째 연결: 가중치 64 x 32 = 2048개, 편향 32개
2번째 연결: 가중치 32 x 10 = 320개, 편향 10개
가중치와 편향을 합한 전체 개수: 2410개
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

```
=== 실험 1. 64칸의 순서를 섞어 새로 학습 ===
원래 순서로 학습                0.933
섞은 순서로 학습                0.943
차이: +0.009

=== 실험 2. 테스트 이미지를 밀어서 넣기 (다시 학습하지 않는다) ===
조건                      테스트 정확도
원본                        0.933
오른쪽 1칸                   0.439
왼쪽 1칸                    0.376
아래 1칸                    0.470
오른쪽 2칸                   0.152
기준선: 0.102
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

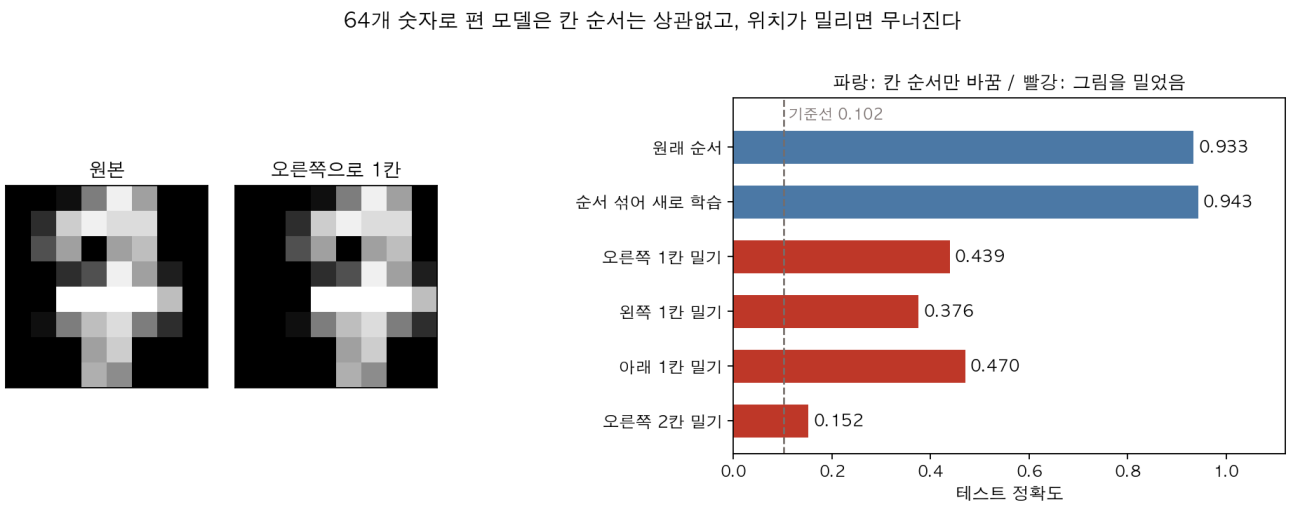


그림 9.5 왼쪽은 원본과 오른쪽으로 1칸 민 이미지다. 오른쪽 막대에서 파랑은 칸 순서만 바꾼 경우, 빨강은 그림을 민 경우의 테스트 정확도다. 점선은 기준선이다

표 9.2 자주 헷갈린 짝 (테스트 540장 기준)

정답	모델 예측	건수
8	1	8
9	7	4
1	9	4
1	8	4
5	9	3
2	1	2

출처: `practice/chapter9/results/9-1-flatten-classifier.log`



그림 9.6 실패 갤러리. 각 그림 위에 정답과 모델의 예측을 적었다

결과 읽기

○ 평탄화해도 0.933까지 맞힌다

- 기준선 0.102는 가장 많은 숫자 하나로만 답했을 때의 정확도다. 정답이 열 가지이고 개수가 고르므로 10분의 1 근처가 나온다
- 테스트 0.933은 기준선의 아홉 배가 넘는다. 3절의 문제가 있어도 이 데이터에서 평탄화 모델이 쓸모없지는 않다
- 훈련 0.960과 테스트 0.933의 차이는 0.027이다. 7장에서 본 과적합의 간격이 작은 경우다

○ 예측 1이 맞았다: 순서를 섞어도 정확도가 거의 같다

- 원래 순서 0.933, 섞은 순서 0.943이다. 섞은 쪽이 오히려 조금 높다
- 두 값이 완전히 같지 않은 이유는 학습 시작 때 가중치를 무작위로 정하기 때문이다. 같은 seed라도 칸 번호가 바뀌면 각 칸에 배정되는 처음 가중치가 달라진다
- 이 결과는 평탄화 모델이 "어느 칸이 어느 칸 옆인가"를 쓰지 않는다는 증거다. 쓰고 있었다면 순서를 섞은 뒤 성능이 떨어졌어야 한다

○ 예측 2도 맞았다: 한 칸만 밀어도 정확도가 반 아래로 떨어진다

- 오른쪽 1칸에서 0.439, 왼쪽 1칸에서 0.376, 아래 1칸에서 0.470이다. 원본 0.933의 절반에도 못 미친다
- 오른쪽 2칸이면 0.152로 기준선 0.102에 가까워진다
- 밀어서 잘려 나간 밝기는 원인이 아니다. 오른쪽 1칸 밀 때 잘려 나간 밝기 합은 이미지 한 장 평균 1.1이고, 전체 밝기 합 평균은 312.2다. 획은 거의 그대로 남았고 위치만 바뀌었다

○ 틀린 이미지에는 규칙이 있다

- 8을 1로 본 것이 8건으로 가장 많다. 그림 9.6의 두 번째 줄 두 번째 그림이 그 예다. 8 x 8까지 줄인 8은 가운데 세로 획만 굵게 남는 경우가 있다
- 1·8·9·7이 서로 섞인다. 1을 9로 4건, 1을 8로 4건, 9를 7로 4건 틀렸다. 네 숫자 모두 가운데에 긴 세로 획이 있다
- 표 9.2에서 보듯 틀린 건수는 특정 짝에 몰려 있다. 정확도 0.933이라는 숫자 하나로는 이 사실이 보이지 않는다. 4장에서 혼동행렬을 배운 이유가 여기서 다시 나온다

학습 중 나온 경고

로그에 경고가 하나 찍힌다. 지우지 않고 그대로 뒀다.

```
[ConvergenceWarning] Stochastic Optimizer: Maximum iterations (60) reached and the optimiza
```

출처: `practice/chapter9/results/9-1-flatten-classifier.log`

`MAX_ITER = 60`으로 작게 잡아서 나온 경고다. 학습이 아직 끝나지 않았는데 반복 횟수를 다 썼다는 뜻이다. 수업 시간 안에 끝나는 것을 우선해 이 값을 작게 뒀다.

해 보기

1. `SAMPLE_INDEX`를 0, 1, 2로 바꿔 다른 숫자의 픽셀 표를 본다. 0과 1은 숫자 64개의 배치가 어떻게 다른가
2. `MAX_ITER`를 300으로 올린다. 경고가 사라지는가. 원본 정확도와 1칸 민 정확도는 각각 얼마가 되는가
3. `HIDDEN_SIZE`를 128로 올린다. 모델을 키우면 1칸 민 이미지의 정확도도 회복되는가
4. `GALLERY_SIZE`를 20으로 올려 실패 갤러리를 넓게 본다. 8을 1로 본 이미지가 몇 장 들어오는가

○ 2번과 3번은 같은 질문이다. **학습을 더 하거나 모델을 키우는 것으로 위치 문제가 풀리는가.** 결과를 확인한 뒤 4절로 넘어간다

---

4. 합성곱과 필터

평탄화 모델은 칸 번호마다 따로 가중치를 배운다. 실습 9.1의 실험 2에서 그림을 한 칸 밀자 정확도가 0.933에서 0.439로 떨어진 결과도 이 구조와 맞는다. 합성곱은 다른 방식으로 묻는다. "이 근처 3 x 3 칸에 세로 획 모양이 있는가"를 이미지의 **모든 위치에서 같은 방식으로** 묻는다.

4-1. 창 하나에서 곱하고 더한다

필터(filter)는 작은 숫자 판이다. 보통 3 x 3이나 5 x 5를 쓴다. **합성곱(convolution)**은 필터를 이미지 위에서 한 칸씩 옮기며, 겹친 숫자끼리 곱해서 전부 더하는 계산이다.

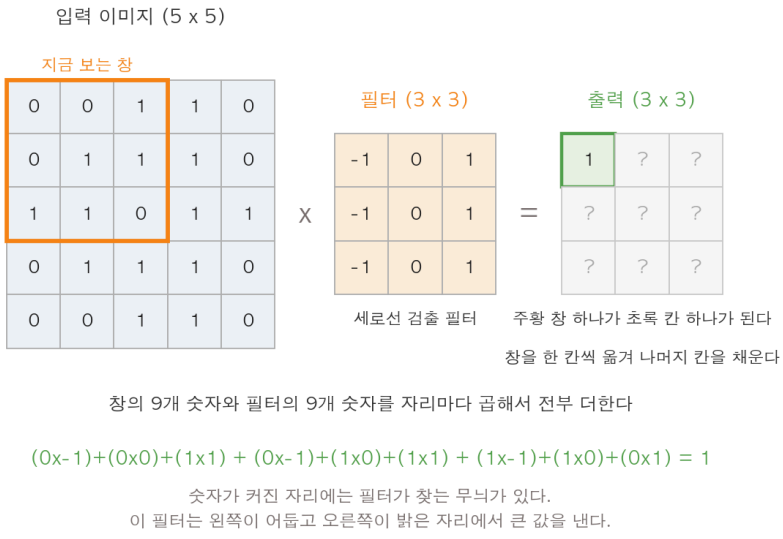


그림 9.7 주황 창 안의 숫자 아홉 개와 필터의 숫자 아홉 개를 위치마다 곱해 더하면, 출력 칸 하나가 채워진다. 그림의 5 x 5 입력은 설명용으로 손으로 고른 값이다

계산 순서는 세 단계다.

1. 이미지 왼쪽 위에서 3 x 3 창을 하나 떼어 낸다

2. 창의 숫자 아홉 개와 필터의 숫자 아홉 개를 **위치마다** 곱한 뒤 전부 더한다
3. 그 합 하나를 출력 칸 하나에 적고, 창을 오른쪽으로 한 칸 옮겨 반복한다

실습 9.2는 이 계산을 로그에 그대로 풀어 찍는다. 아래는 1절의 17번 이미지에서 2행 2열부터 시작하는 창에 세로선 필터를 씌운 결과다.

```
이미지 창            필터              곱한 값

  10    0   10    x   -1.0   0.0   1.0   =   -10.0    0.0   10.0
   3    5   15    x   -1.0   0.0   1.0   =    -3.0    0.0   15.0
  16   16   16    x   -1.0   0.0   1.0   =   -16.0    0.0   16.0

더하기: (10x-1) + (0x0) + (10x1) + (3x-1) + (5x0) + (15x1) + (16x-1) + (16x0) + (16x1)
      = 12.0
```

출처: `practice/chapter9/results/9-2-filter-and-pooling.log`

4-2. 필터는 생김새가 닮은 창에 큰 값을 낸다

세로선 필터는 왼쪽 열이 -1, 가운데 열이 0, 오른쪽 열이 +1이다. 가운데 열에는 0을 곱하므로 결과는 다음과 같다.

```
세로선 필터의 결과 = (오른쪽 열 세 칸의 합) − (왼쪽 열 세 칸의 합)
```

위 계산에 넣어 보면 오른쪽 열 합은 10 + 15 + 16 = 41, 왼쪽 열 합은 10 + 3 + 16 = 29이고, 41 − 29 = 12다. 곱셈 아홉 개를 푼 결과와 같다.

이 식에서 원리가 나온다. 필터의 +1 칸에 밝은 픽셀이, -1 칸에 어두운 픽셀이 겹치면 결과가 커진다. 즉 **창의 밝기 배치가 필터의 숫자 배치와 닮을수록 결과가 크다.** 반대로 생긴 창은 큰 음수가 되고, 좌우 밝기가 비슷한 창은 0 근처가 된다. 실습 9.2는 17번 이미지의 36개 창 가운데 세 개를 골라 이것을 확인한다.

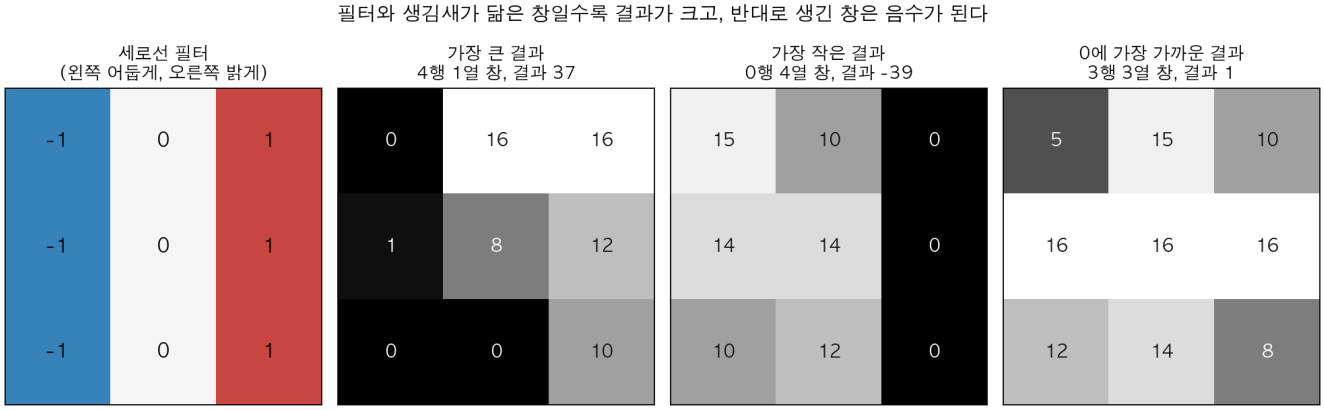


그림 9.8 맨 왼쪽은 세로선 필터다. 나머지 세 장은 17번 이미지에서 이 필터의 결과가 가장 큰 창, 가장 작은 창, 0에 가장 가까운 창이다

```
가장 큰 결과: 4행 1열 창, 결과 37
  왼쪽 열 합 1, 오른쪽 열 합 38, 오른쪽 - 왼쪽 = 37
가장 작은 결과: 0행 4열 창, 결과 -39
  왼쪽 열 합 39, 오른쪽 열 합 0, 오른쪽 - 왼쪽 = -39
0에 가장 가까운 결과: 3행 3열 창, 결과 1
  왼쪽 열 합 33, 오른쪽 열 합 34, 오른쪽 - 왼쪽 = 1
```

출처: `practice/chapter9/results/9-2-filter-and-pooling.log`

○ **결과 37인 창** — 왼쪽 열이 거의 검정(합 1)이고 오른쪽 열이 밝다(합 38). 필터의 -1·+1 배치와 같은 모양이다. 획이 시작되는 왼쪽 경계다

○ **결과 -39인 창** — 왼쪽 열이 밝고(합 39) 오른쪽 열이 검정(합 0)이다. 필터와 좌우가 뒤집힌 모양이다. 획이 끝나는 오른쪽 경계다

○ **결과 1인 창** — 가운데 줄이 16, 16, 16으로 꽉 찼다. 좌우 밝기가 33과 34로 거의 같아서 세로 경계가 없다

필터를 바꾸면 "무엇과 닮았는가"의 기준이 바뀐다. 위 줄이 -1이고 아래 줄이 +1인 필터는 위가 어둡고 아래가 밝은 창, 즉 가로 경계에 큰 값을 낸다.

4-3. 창을 옮기며 판 전체를 채운다

창 하나가 출력 칸 하나를 채운다. 창을 왼쪽 위에서 오른쪽으로 한 칸씩 옮기고, 줄 끝에 닿으면 한 줄 내려가 다시 왼쪽부터 옮긴다.

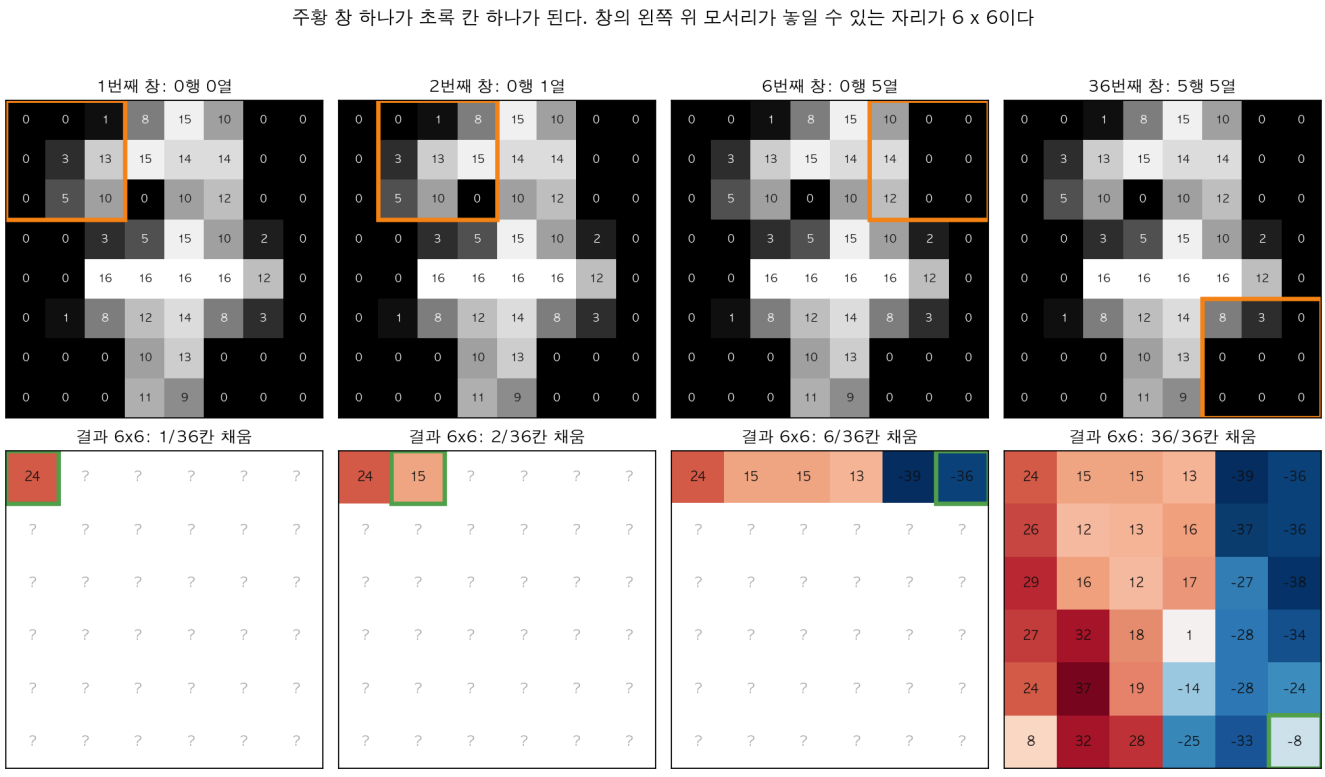


그림 9.9 윗줄의 주황 창 하나가 아랫줄의 초록 칸 하나가 된다. 1번째·2번째·6번째·36번째 창을 골라 그렸다

○ **8 x 8이 6 x 6이 되는 이유**

- 창은 이미지 밖으로 나갈 수 없다. 3 x 3 창의 왼쪽 위 모서리가 놓일 수 있는 위치는 가로로 0~5열의 6개, 세로로 0~5행의 6개다
- 그래서 출력은 6 x 6 = 36칸이다. 일반식은 `출력 크기 = 입력 크기 - 필터 크기 + 1`이고, 8 - 3 + 1 = 6이다
- 가장자리를 0으로 채워 출력 크기를 입력과 같게 유지하는 방법(padding)도 있다. 이 장의 계산은 padding 없이 한다

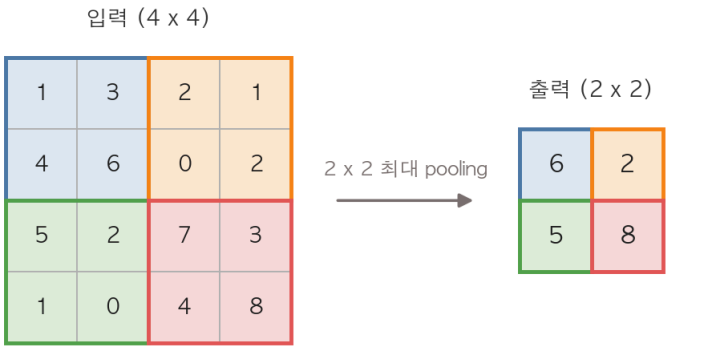
○ **36개 창 모두에 같은 필터를 쓴다**

- 평탄화 모델은 64칸마다 다른 가중치를 두었다. 합성곱은 36개 위치 모두에 같은 숫자 아홉 개를 쓴다
- 그래서 어떤 무늬가 2열에 있든 4열에 있든 같은 크기의 값이 나온다. 무늬가 옮겨 가면 큰 값이 나오는 출력 칸도 함께 옮겨 갈 뿐이다. 실습 9.2에서 그림을 한 칸 밀어 이것을 확인한다
- 이미지가 커져도 필터의 숫자는 아홉 개 그대로다. 3절의 문제 3(가중치 폭증)이 이렇게 줄어든다

---

5. pooling — 크기를 줄이고 강한 반응만 남긴다

합성곱 출력은 입력보다 조금 작을 뿐 여전히 숫자가 많다. **pooling**은 출력 크기를 크게 줄이는 단계다. 가장 흔한 방식은 겹치지 않는 2 x 2 덩어리마다 가장 큰 값 하나만 남기는 **최대 pooling(max pooling)**이다.



덩어리마다 가장 큰 숫자 하나만 남긴다. 숫자 16개가 4개로 줄었다.

파란 덩어리의 1, 3, 4, 6 중에서 6을 고른다

자리가 한두 칸 흔들려도 가장 큰 값은 잘 바뀌지 않는다.
그래서 pooling 뒤에는 위치가 조금 달라진 그림도 비슷한 숫자가 나온다.

그림 9.10 2 x 2 덩어리마다 최댓값 하나만 남긴다. 숫자 16개가 4개로 줄었다. 그림의 숫자는 설명용으로 손으로 고른 값이다

○ **크기가 4분의 1로 줄어든다**

- 6 x 6에 2 x 2 pooling을 하면 3 x 3이 된다. 가로 절반, 세로 절반이므로 칸 수는 36에서 9로 4분의 1이 된다
- 뒤에 오는 층이 다룰 숫자가 그만큼 줄어든다

○ **덩어리 안에서의 작은 이동은 지운다**

- pooling 결과에는 덩어리 안 네 칸 가운데 어느 칸이 가장 컸는지가 남지 않는다
- 그래서 강한 반응이 덩어리 안에서 한 칸 옮겨 가면 pooling 결과는 그대로다

○ 덩어리 경계를 넘는 이동은 지우지 못한다

- 강한 반응이 옆 덩어리로 넘어가면 그 값은 옆 칸에 나타난다
- 2 x 2 pooling 한 번으로 모든 이동을 흡수하지는 못한다. 실습 9.2에서 이 경우를 직접 본다

○ 남는 것과 잃는 것

- "얼마나 강한 반응이었는가"는 남고, "정확히 어느 칸이었는가"는 흐려진다
- 위치를 정밀하게 알아야 하는 문제에서는 이 손실이 단점이 된다

○ 바로 다음 실습 9.2의 설정

- 1절의 17번 이미지를 그대로 쓴다
- 창 하나의 계산(4-1), 닮은 창 찾기(4-2), 창 옮기기(4-3)를 로그와 그림으로 출력한다
- 필터 네 개를 적용해 결과를 비교하고, 내가 만든 필터를 하나 넣어 본다
- 이미지를 오른쪽으로 한 칸 민 뒤, 합성곱 결과와 2 x 2 pooling 결과를 원본과 비교한다

실습 9.2: 필터와 pooling을 직접 계산한다

실습 목표

- 3 x 3 창 하나의 합성곱을 곱셈 아홉 개까지 풀어서 확인한다
- 필터의 숫자를 직접 바꿔 결과가 달라지는 것을 본다
- 그림을 한 칸 밀었을 때 합성곱 결과와 pooling 결과가 어떻게 바뀌는지 확인한다

데이터

`load_digits()`의 8 x 8 손글씨 1797장 중 한 장을 고른다. 기본값은 1절의 17번 이미지다.

실행 명령

```
cd practice/chapter9/code
python 9-2-filter-and-pooling.py
```

핵심 코드

파일 맨 위에 학생이 바꿀 값이 세 개 있다.

```
SAMPLE_INDEX = 17           # 몇 번째 손글씨를 볼 것인가
MY_FILTER = np.array([      # 내가 만드는 필터. 3x3 숫자를 마음대로 바꿔 본다
    [0.0, -1.0, 0.0],
    [-1.0, 4.0, -1.0],
    [0.0, -1.0, 0.0],
])
SHIFT_PIXELS = 1            # pooling 비교에서 이미지를 오른쪽으로 몇 칸 밀 것인가
```

합성곱은 반복문 두 개다. 창을 떼어 내고, 필터와 곱해 더한다.

```
for r in range(out_h):
    for c in range(out_w):
        window = image[r:r + kh, c:c + kw]
        out[r, c] = float(np.sum(window * kernel))
```

최대 pooling도 반복문 두 개다. 2 x 2 덩어리를 떼어 내고, 가장 큰 값을 남긴다.

```
for r in range(out_h):
    for c in range(out_w):
        block = feature[r * size:(r + 1) * size, c * size:(c + 1) * size]
        out[r, c] = float(block.max())
```

전체 코드는 `practice/chapter9/code/9-2-filter-and-pooling.py` *참고*

실행 결과

창 하나의 계산과 닮은 창 세 개는 4절에 옮겨 두었다. 필터 네 개의 결과는 다음과 같다.

표 9.3 필터 네 개를 8 x 8 이미지에 적용한 결과

필터	3x3 숫자	출력 크기	최솟값	최댓값
세로선 검출	왼쪽 열 -1, 오른쪽 열 +1	6x6	-39.0	37.0
가로선 검출	위 줄 -1, 아래 줄 +1	6x6	-31.0	28.0
흐리게	아홉 칸 전부 1/9	6x6	1.0	12.4
내 필터(기본값)	가운데 4, 상하좌우 -1	6x6	-40.0	37.0

출처: `practice/chapter9/results/9-2-filter-and-pooling.log`

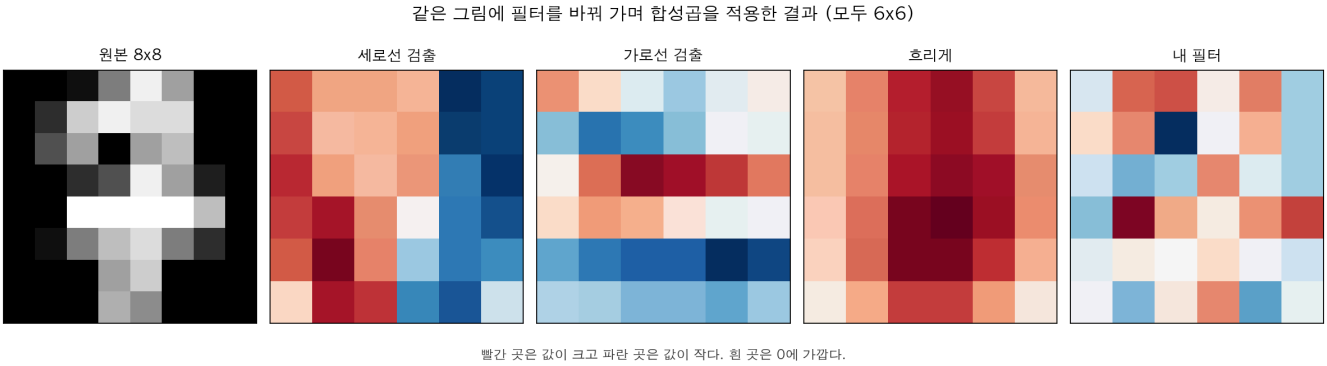


그림 9.11 같은 그림에 필터만 바꿔 합성곱을 적용한 결과. 빨간 칸은 값이 크고 파란 칸은 값이 작다

이미지를 오른쪽으로 한 칸 밀고 세로선 필터와 2 x 2 pooling을 다시 계산한 결과는 다음과 같다.

```
원본 결과의 2x2 최대 pooling 3x3:
  0 |   26   16  -36
  1 |   32   18  -27
  2 |   37   28   -8
밀린 결과의 2x2 최대 pooling 3x3:
  0 |   26   15   16
  1 |   29   32   17
  2 |   24   37  -14
3x3 아홉 칸 중 값이 같은 칸: 1칸
가장 큰 반응: 원본 37, 밀린 이미지 37
```

출처: `practice/chapter9/results/9-2-filter-and-pooling.log`

그림을 한 칸 밀면 합성곱 결과도 같은 숫자 그대로 한 칸 밀린다 (검은 선은 pooling 덩어리)

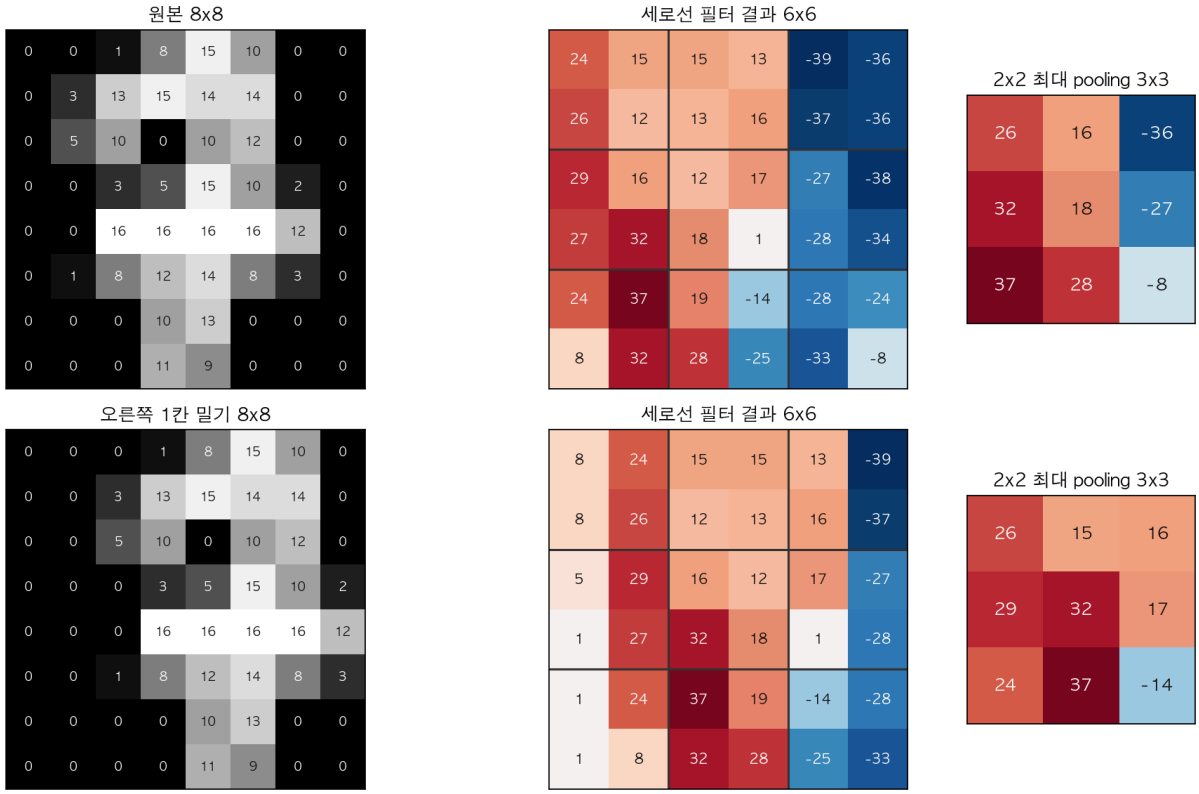


그림 9.12 윗줄은 원본, 아랫줄은 오른쪽으로 1칸 민 이미지다. 가운데 판은 세로선 필터 결과 6 x 6이고, 검은 선은 2 x 2 pooling 덩어리의 경계다. 오른쪽 판은 pooling 결과 3 x 3이다

결과 읽기

○ 흐리게 필터만 최솟값이 양수다

- 아홉 칸이 전부 1/9라서 결과는 주변 아홉 칸의 평균이다. 평균은 원래 값의 범위(0~16)를 벗어나지 않는다
- 나머지 세 필터에는 음수가 들어 있어서 결과도 음수가 나온다. 음수는 "필터가 찾는 무늬와 반대로 생긴 창"이라는 뜻이다(4-2)

○ 그림을 한 칸 밀면 합성곱 결과도 같은 값 그대로 한 칸 밀린다

- 로그에는 "밀린 결과의 1열부터 끝까지 == 원본 결과의 0열부터 4열까지: 같다"가 찍힌다. 그림 9.12 가운데 판의 24, 26, 29, 27, 24, 8 열이 한 칸 오른쪽으로 옮겨 갔다
- 실습 9.1의 평탄화 모델은 같은 이동에서 정확도가 0.933에서 0.439로 떨어졌다. 합성곱 결과는 값이 바뀌지 않고 위치만 바뀐다
- 36개 위치 모두에 같은 필터를 썼기 때문이다(4-3)

○ 2 x 2 pooling 한 번으로는 한 칸 이동을 다 흡수하지 못한다

- pooling 결과 아홉 칸 가운데 값이 같은 칸은 1칸뿐이다
- 가장 큰 반응 37은 원본에서 왼쪽 아래 덩어리에 있었고, 밀린 이미지에서는 가운데 아래 덩어리로 넘어갔다. 덩어리 경계를 넘은 이동이다(5절)

- 그래도 가장 큰 반응의 크기 37은 그대로 남았다. pooling을 여러 번 거쳐 덩어리가 커질수록, 이미지 안 어디에 무늬가 있든 "그 무늬가 있었다"는 사실이 같은 값으로 남는다

해 보기

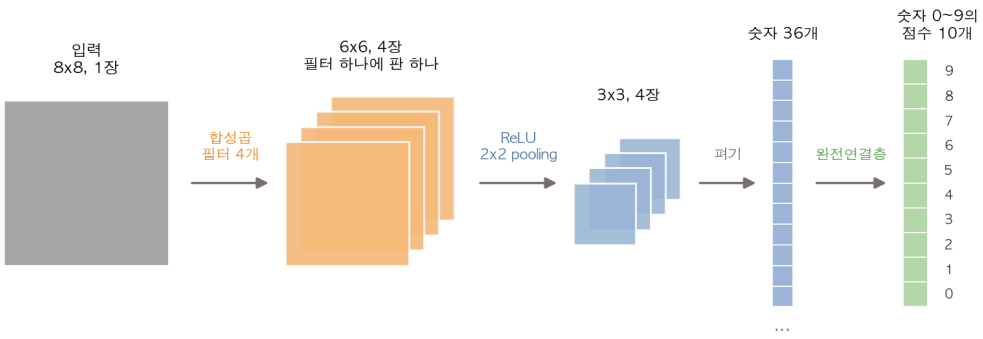
1. `MY_FILTER`의 아홉 개 숫자를 전부 0으로 둔다. 결과가 어떻게 되는가
2. 세로선 필터를 90도 돌린 값(위 줄 -1, 아래 줄 +1)을 `MY_FILTER`에 넣는다. 가로선 검출 필터와 같은 그림이 나오는가
3. 가운데만 1이고 나머지가 0인 필터를 넣으면 원본이 그대로 나온다. 왜 그런지 4-1의 계산 순서로 설명한다
4. `SHIFT_PIXELS`를 2로 바꾼다. pooling 결과 3 x 3에서 값이 같은 칸이 몇 칸으로 바뀌는가. 두 칸 이동이 2 x 2 덩어리와 어떤 관계인지 설명한다
5. `SAMPLE_INDEX`를 0으로 바꾼다. 숫자 0의 세로선 필터 결과에서 양수와 음수는 각각 어느 쪽에 모이는가

○ 확인할 것은 정확도가 아니다. **필터의 숫자 아홉 개를 바꾸면 찾는 무늬가 바뀐다**는 것과 **합성곱 결과는 그림과 함께 옮겨 간다**는 것을 눈으로 확인하면 된다

---

6. CNN의 전체 모양

CNN(Convolutional Neural Network, 합성곱 신경망)은 4~5절의 계산을 층으로 쌓은 신경망이다.



판 크기는 8 -〉 6 -〉 3으로 줄고, 판 장수는 1 -〉 4로 는다. 학습으로 찾는 값은 필터 4개의 숫자 36개와 완전연결층의 가중치다.

이 그림은 층 순서와 판 크기를 보여 주는 개념 도식이다.

그림 9.13 8 x 8 입력에 필터 4개를 쓰는 작은 CNN의 개념 도식이다. 판의 크기는 8 → 6 → 3으로 줄고, 판의 장수는 1 → 4로 는다

그림 9.13의 크기는 이 장의 계산 규칙에서 나온다. 3 x 3 필터를 쓰면 8 x 8이 6 x 6이 되고(4-3), 필터 하나가 판 하나를 만들므로 필터 4개면 6 x 6 판이 4장이다. 2 x 2 pooling을 거치면 3 x 3 판 4장이 되고, 이것을 펴면 숫자 36개다. 마지막 완전연결층이 이 36개로 숫자 0~9의 점수 10개를 계산한다. 실제 CNN은 합성곱·ReLU·pooling 묶음을 여러 번 반복하고, 필터도 수십~수백 개를 쓴다.

○ **앞쪽 층은 단순한 무늬를, 뒤쪽 층은 무늬의 조합을 찾는다**

- 첫 합성곱 층의 필터는 4절에서 본 것처럼 경계나 선에 큰 값을 낸다
- 그 결과 판 위에 다시 합성곱을 걸면, 다음 층의 필터는 "세로 경계 판과 가로 경계 판에서 이런 배치로 큰 값이 모인 곳"에 큰 값을 낸다
- pooling으로 판 크기를 줄여 가며 이 과정을 반복하면, 뒤쪽 층의 칸 하나가 입력 이미지의 넓은 범위를 대표하게 된다

○ 필터의 숫자를 사람이 정하지 않는다

- 실습 9.2에서 필터의 숫자 아홉 개는 사람이 정했다. 세로선 필터도, 흐리게 필터도 사람이 고른 값이다
- CNN에서는 이 아홉 개 숫자가 **가중치**다. 6장의 경사하강법이 손실을 줄이는 방향으로 이 숫자를 고친다
- 즉 CNN이 학습하는 것은 "어떤 무늬를 찾을지"다

표 9.4 3절의 문제와 CNN이 쓰는 방법

평탄화 방식의 문제	실습에서 확인한 것	합성곱·pooling이 하는 일
칸 사이의 이웃 관계를 쓰지 않는다	순서를 섞어도 0.933 → 0.943	3 x 3 창으로 붙어 있는 칸끼리만 묶어 계산한다
위치가 밀리면 배운 것을 쓰지 못한다	1칸 밀면 0.933 → 0.439	같은 필터를 모든 위치에 쓰므로 결과가 그림과 함께 옮겨 가고, pooling이 작은 이동을 지운다
이미지가 커지면 가중치가 폭증한다	8 x 8에서도 첫 층 가중치 2,048개	필터 하나의 숫자는 9개이고, 이미지가 커져도 그대로다

출처: `practice/chapter9/results/9-1-flatten-classifier.log`, `practice/chapter9/results/9-2-filter-and-pooling.log`

두 실습으로 확인한 것은 두 가지다. 평탄화 모델은 그림이 한 칸만 밀려도 정확도가 반 아래로 떨어진다(실습 9.1). 사람이 정한 필터 하나는 이미지에서 특정 무늬를 끄집어내고, 그림이 밀리면 그 결과도 같은 값으로 함께 밀린다(실습 9.2). 표 9.4의 오른쪽 열은 합성곱과 pooling의 계산 구조에서 나오는 성질이며, 학습을 마친 CNN의 정확도로 확인한 값이 아니다.

필터의 숫자를 데이터에서 찾는 원리는 6장의 경사하강법과 같다. 실제로 CNN을 학습시킬 때는 합성곱 층과 경사 계산을 함께 제공하는 PyTorch나 TensorFlow 같은 딥러닝 라이브러리를 쓴다.

---

7. 직접 움직여 보는 웹 자료

수업에서 다 보여줄 수 없는 그림은 아래 자료로 보충한다. 모두 무료이고, 아래 설명은 각 페이지를 열어 확인한 내용이다.

자료	무엇을 볼 수 있는가	주소
Setosa, Image Kernels	얼굴 사진 위에서 커널을 고르거나 3x3 숫자를 직접 넣으면 결과가 바로 바뀐다. 픽셀에 마우스를 올리면 그 칸의 계산식이 나온다	https://setosa.io/ev/image-kernels/
Google ML 실습(한국어), 컨볼루셔널 신경망	5x5 위를 3x3 창이 훑는 애니메이션과 2x2 최대 풀링 애니메이션을 나란히 본다	https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks?hl=ko
vdumoulin, conv_arithmetic	창을 옮기는 폭(stride)과 가장자리 채우기(padding)를 바꿨을 때 출력 크기가 어떻게 달라지는지 애니메이션 15개 이상으로 비교한다	https://github.com/vdumoulin/conv_arithmetic
scikit-learn, Recognizing hand-written digits	실습 9.1과 같은 8x8 손글씨 1797장을 SVM으로 분류하고 혼동행렬을 그린다	https://scikit-learn.org/stable/auto_examples/classification/plot_digits_classification.html

○ Setosa의 커널 실습은 실습 9.2와 짝이 된다. `MY_FILTER`를 바꾸기 전에 이 페이지에서 먼저 숫자를 바꿔 보면, 어떤 값이 어떤 무늬를 도드라지게 하는지 미리 확인할 수 있다

○ Google ML 실습 페이지에는 "2025년 12월 15일에 삭제될 예정"이라는 지원 중단 공지가 붙어 있다. **그 날짜는 이미 지났는데 확인 시점(2026년 8월)에는 아직 열렸다.** 안 열리면 conv_arithmetic의 첫 번째 애니메이션으로 대신한다

○ conv_arithmetic에는 전치 합성곱(transposed convolution)처럼 이 장에서 다룬 내용보다 앞선 주제의 그림도 함께 있다. 첫 묶음의 "no padding, no strides" 하나만 보면 된다. MIT 라이선스다

과제 (LMS 제출)

수업 시간에 실습을 실행했는지만 확인한다. 따로 보고서를 쓰지 않는다.

제출물

두 실습을 실행하면 만들어지는 그림 파일 두 장을 LMS에 올린다.

1. `practice/chapter9/results/shift_test.png`
2. `practice/chapter9/results/filter_gallery.png`

평가 기준

- 두 파일을 올리면 완료로 인정한다. 정확도가 얼마인지는 채점하지 않는다
- 상수를 바꿨다면 바뀐 그림이 올라온다. 기본값 그대로여도 인정한다

마감

- 실습 수업일 포함 7일 이내

핵심 정리

개념	한 줄 정리
픽셀	이미지의 가장 작은 칸 하나. 흑백에서는 숫자 하나가 밝기를 뜻한다
채널	같은 크기의 숫자 판이 몇 장 겹쳤는가. 흑백은 1장, 컬러는 3장이다
평탄화	이미지를 한 줄로 펴는 것. 칸 순서를 섞어도 성능이 같고, 그림을 1칸 밀면 성능이 반 아래로 떨어진다
합성곱	작은 창을 옮기며 겹친 숫자끼리 곱해 더하는 계산. 8x8에 3x3을 쓰면 6x6이 나온다
필터	합성곱에 쓰는 작은 숫자 판. 창의 밝기 배치가 필터의 숫자 배치와 닮을수록 결과가 크다
같은 필터 공유	모든 위치에 같은 필터를 쓰므로, 그림이 옮겨 가면 결과도 같은 값으로 옮겨 간다
pooling	덩어리마다 최댓값 하나만 남긴다. 크기를 줄이고 덩어리 안의 작은 이동을 지운다
CNN	합성곱과 pooling을 층으로 쌓은 신경망. 필터의 숫자를 데이터에서 학습한다

다음 장에서는 이미지 대신 문장을 다룬다. 문장을 숫자로 바꾸는 방법이 이미지와 어떻게 다른지가 드러난다.